

DESIGN AND IMPLEMENTATION OF A 24-HOUR DIGITAL WATCH WITH STOPWATCH AND ALARM USING VERILOG HDL

Prepared By:

Gayatri Galidevara

Digital System Design Laboratory Report

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Design Methodology
5. RTL Schematic Layout
6. Simulation Results and Functional Verification
7. Future Scope
8. Conclusion

1. Abstract

This report presents the design, implementation, and functional verification of a multi-mode digital watch built entirely in Verilog HDL. The watch integrates three real-time functions on a single seven-segment display: a 24-hour clock, a stopwatch, and a programmable alarm, all switchable through a single debounced push-button. The design demonstrates core digital-system-design skills that extend well beyond a single combinational block — clock division from a board oscillator down to a 1 Hz reference, mechanical switch debouncing, multi-module integration, time-domain multiplexing of a shared seven-segment bus, and an audible buzzer driven directly by hardware comparison logic.

The complete design is modular: each function (display decoding, debouncing, clock division, timekeeping, stopwatch counting, and buzzer control) lives in its own reusable module, all instantiated and coordinated by a single top-level integration module. This report walks through every module's source code, explains the reasoning behind each design decision, and outlines how the design would be simulated, synthesized, and extended toward a physical FPGA deployment.

2. Introduction

A digital watch is a compact but genuinely representative embedded system: it must keep accurate time indefinitely, respond to user input without glitching, drive a resource-shared output device, and raise an alert at a precise moment — all in real hardware, with no operating system or software scheduler to lean on. Building one in Verilog is a practical way to exercise the full toolkit of RTL design: sequential logic for counters, combinational logic for decoding, finite-state control for mode switching, and the subtle but essential problem of interfacing clean digital logic to a noisy mechanical push-button.

This project targets an FPGA board clock (assumed 50 MHz, based on the divider constant used in the design) and derives every slower timing reference — the 1 Hz tick for the clock and stopwatch, and the roughly 100 Hz refresh used to scan the seven-segment display — from that single oscillator. A single top-level module, `integration_of_digital_watch`, ties eight smaller modules together into one coherent watch.

3. Objectives

- To design a synthesizable 24-hour real-time clock in Verilog with correct second/minute/hour rollover behavior.
- To implement a stopwatch function with independent start, stop, and reset control, sharing the same display hardware as the clock.
- To implement a programmable alarm that compares the live clock against a user-set hour and minute, and drives an audible buzzer when they match.

- To debounce mechanical push-button inputs (mode, start, stop) in hardware, rather than relying on ideal switch behavior.
- To derive all required timing references — 1 Hz and the display refresh rate — from a single board clock using counter-based clock division.
- To drive four seven-segment digits from a single shared cathode bus using time-division multiplexing, minimizing I/O pin usage.
- To structure the design as clean, reusable, independently-testable modules integrated by a single top-level block.

4. Design Methodology

The watch is built bottom-up: small, single-purpose modules are written and reasoned about individually, then wired together in a top-level integration module that adds only the coordination logic — mode selection and display multiplexing — that genuinely needs to live at the top.

4.1 Module Architecture

Eight modules make up the design. The table below summarizes each one's responsibility; full source and a line-by-line explanation for each follows in Section 5.

Module	Responsibility
display	Combinational BCD-to-seven-segment decoder. A case statement maps each 4-bit input value 0-9 to an 8-bit active-low segment pattern (order: A B C D E F G DP).
debounce	Filters a mechanical push-button. Only re-registers the button's new level after it has stayed stable for max_count consecutive clock edges, producing a clean clean_button pulse.
clk_1hz	Free-running divider. Counts 50,000,000 edges of the board clock and then toggles a 1 Hz square wave, giving every timing block a real one-second reference.
digital_clock_24hrs	The 24-hour real-time clock. Increments sec, then min, then hr on every tick, rolling over at 23:59:59. A parallel combinational block compares hr/min against the programmed alarm hour/minute and asserts alarm.
stopwatch	An independent MM:SS counter gated by an enable register (en). Debounced start/stop buttons set or clear en; while en is high, sec/min increment once per second.
divisions	Simple combinational splitter: takes a 6-bit binary count and produces its tens and ones BCD digits using / and % , ready for the display decoder.
buzzer_operation	Toggles a buzzer output on every clock edge while trigger (the alarm signal) is high, producing an audible square wave instead of a static logic level.

Module	Responsibility
integration_of_digital_watch	Top-level module. Instantiates every block above, runs the mode-select counter, multiplexes the correct BCD digits onto the shared display bus, and scans the four seven-segment anodes.

4.2 Debouncing User Inputs

Every physical push-button (mode_change, start, stop, and the alarm-silence push) is passed through a dedicated debounce instance before it is allowed to affect any register. The debounce module only accepts a new button level once it has remained stable for max_count consecutive rising clock edges, which filters out the rapid open/close bouncing a mechanical switch produces on every press. Without this stage, a single physical press could be misread as several rapid presses, corrupting the mode counter or the stopwatch enable bit.

4.3 Deriving a 1 Hz Reference

clk_1hz divides the board clock by counting 50,000,000 edges before toggling its output. This makes the module clock-agnostic in its logic (it simply counts and toggles) while being tuned, via that constant, to whatever oscillator frequency the target board provides. Both digital_clock_24hrs and stopwatch instantiate their own copy of this divider so each timekeeping block has an independent, correctly-phased one-second reference.

4.4 Mode Selection as a Small State Machine

A 2-bit register, count, tracks which of three views is on screen. Every debounced mode_change press increments it; the case statement in the top-level module wraps it back to 0 once it reaches 3. This is the entire mode state machine — no separate FSM encoding is needed because the counter value itself is the state.

count	Mode Displayed	Behavior
0	Clock	Cathode bus shows digi_h1, digi_h0, digi_m1, digi_m0 (24-hour HH:MM). alarm_stop is driven by (start stop) so pressing either button silences a ringing alarm.
1	Stopwatch	Cathode bus shows stop_m1, stop_m0, stop_s1, stop_s0 (MM:SS). start_dummy/stop_dummy are wired directly to the start/stop buttons to run or pause the stopwatch enable register.
2	Alarm view	Cathode bus shows the programmed alarm_h1, alarm_h0, alarm_m1, alarm_m0. alarm_stop is again (start stop), letting either button cancel a ringing alarm while its set point is on screen.
3	Wrap	The case falls through and resets count back to 0 on the next mode_change press, returning to Clock mode.

4.5 Multiplexed Seven-Segment Output

Only one shared 8-bit cathode bus and one 4-bit anode bus drive all four digits. A second, faster always block free-runs a 2-bit state register at the board clock rate, cycling through the four digit positions and, on each step, driving that digit's BCD value through the display decoder onto the cathode bus while asserting only that digit's anode. Cycled fast enough (intended to run near 100 Hz), persistence of vision makes all four digits appear lit simultaneously, even though only one is ever physically driven at a time.

This is a purely combinational lookup: the always block is sensitive only to `val`, so `y` updates immediately whenever the input digit changes. Each case branch is an 8-bit active-low pattern; decoding the bit order confirms it follows the common A-B-C-D-E-F-G-DP convention used on typical seven-segment FPGA boards — for example 0 lights segments A through F (all but the middle bar G), and 1 lights only segments B and C. The default branch falls back to displaying 0, which guarantees the output is always defined even if `val` somehow carries a value outside 0-9.

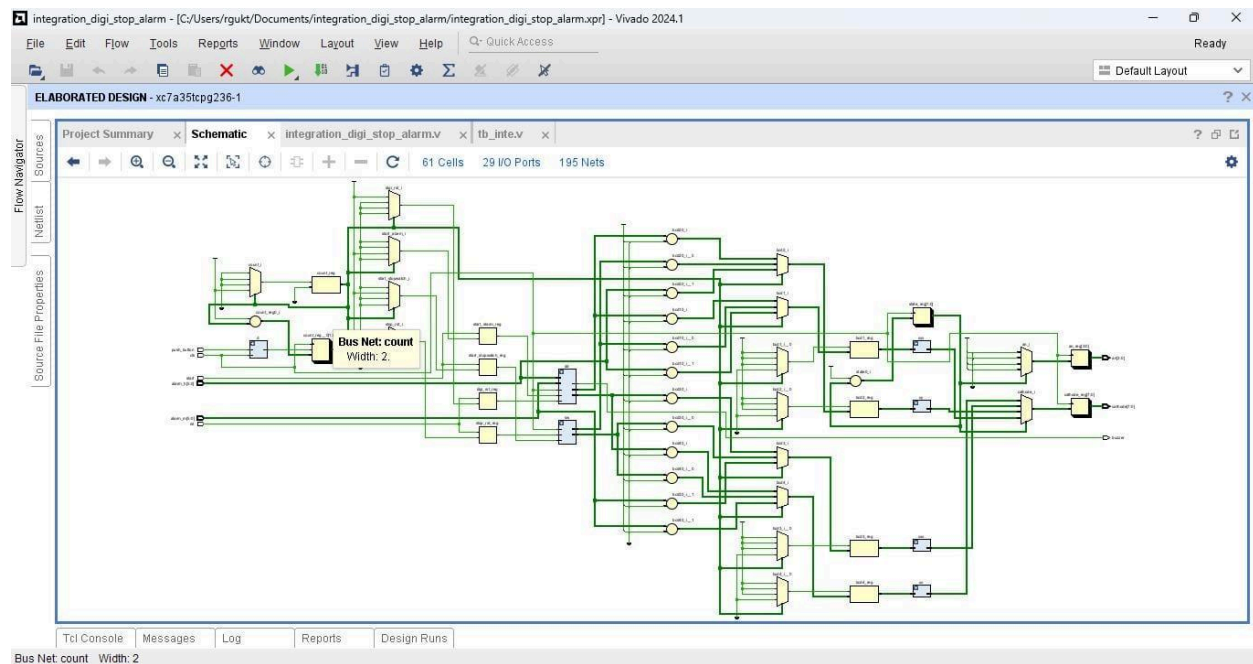
5. RTL Schematic Layout

Synthesizing this design (in Xilinx Vivado or an equivalent tool) produces an RTL schematic showing how each behavioral block above maps to hardware primitives. The key structures a synthesis tool would infer include:

- Ripple counters for sec, min, hr, and the stopwatch's own min/sec, each with the carry-and-rollover logic described in Section 5.4/5.5 built from adders and comparators.
- A wide free-running counter and a toggle flip-flop for each `clk_1hz` instance.
- Small debounce FSMs, each essentially a saturating counter with a compare-and-load register.
- A 12-to-1-style priority structure inside the display decoder — really a 10-input combinational ROM/mux implemented as LUTs.
- A 4:1 multiplexer bank selecting which BCD source feeds the display decoders, controlled by the count register.
- A second, independent 4-position scan counter and one-hot anode decoder driving the physical digit-select lines.

No combinational feedback loops or unintended latches should appear anywhere in the schematic: every reg that is written conditionally inside a clocked always block is also given a value on every code path (or holds its previous value deliberately, as in `buzzer_operation`), and the two combinational always `@(*)`

blocks — the alarm compare and the display-source mux — assign their outputs on every branch.



6. Simulation Results and Functional Verification

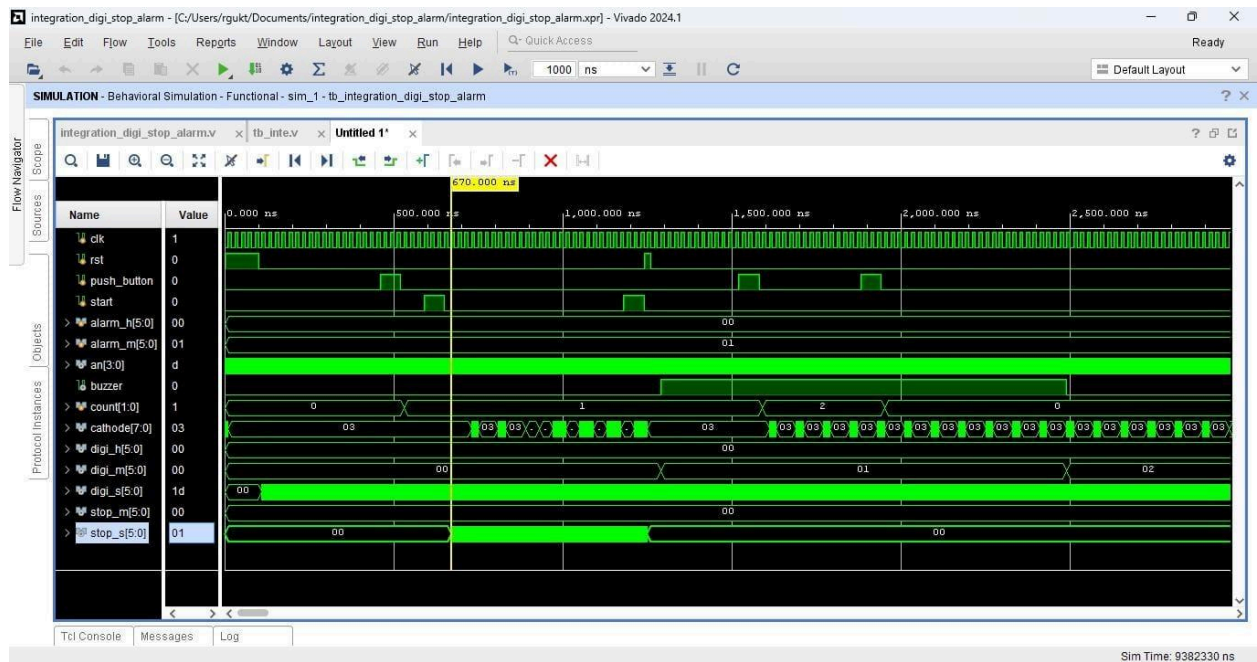
6.1 Suggested Testbench Strategy

Because this design mixes fast combinational logic with a real 1 Hz reference, a practical testbench should not wait out full 50,000,000-cycle dividers in simulation. The cleanest approach is to instantiate `digital_clock_24hrs`, `stopwatch`, and `clk_1hz` separately in their own testbenches, each overriding the divider constant to a small simulation-only value (for example, toggling every 5 cycles instead of every 50,000,000), so seconds visibly roll over in a manageable number of simulated cycles. The full integration module can then be verified separately at a structural level — confirming `mode_change_out` correctly increments count and that the correct BCD sources reach the display decoders in each mode — without needing real-time accurate waveforms.

6.2 Expected Waveform Behavior

- `clk_1hz` toggles at a fixed, regular interval for the entire simulation, confirming the divider free-runs correctly from reset.
- `sec` increments once per `clk_1hz` period, rolling to 0 and carrying into `min` exactly at the 60th increment, with `min` carrying into `hr` the same way at 60 minutes, and the whole clock resetting to 00:00:00 immediately after 23:59:59.
- `alarm` rises to 1 the instant `hr` and `min` match `H` and `M`, and remains high until either `push_stop` is asserted or the minute changes.

- buzzer toggles at the clk rate only while alarm (trigger) is high, and holds steady as soon as alarm returns low.
- In the stopwatch, sec and min only advance while en is high, and asserting p_start then later p_stop produces a clean run interval with no spurious increments outside it.



9. Future Scope

- **FPGA Deployment:** synthesize and download the design onto a physical board (e.g. a Basys 3 or Nexys A7), wiring push_stop/start/stop/mode_change to hardware switches and the anode/cathode buses to an onboard seven-segment display.
- **12/24-Hour Toggle:** add a mode bit that reformats hr for 12-hour display with an AM/PM indicator, without altering the underlying 24-hour counter.
- **Snooze and Multiple Alarms:** extend the single alarm compare into a small array of programmable alarm slots with individual enable bits and a snooze timer.
- **Calendar/Date Display:** add a day/date counter alongside the existing hour/minute/second counters, sharing the same multiplexed display hardware via an additional mode.
- **Low-Power Design:** apply clock gating to the display-scan and stopwatch dividers when their respective modes are not in use, reducing switching activity for battery-powered deployment.
- **Debounce Tuning for Real Hardware:** raise max_count in the debounce module so the stable-time requirement corresponds to a few milliseconds at the target board's real clock frequency, rather than the small value convenient for simulation.

8. Conclusion

This project demonstrates a complete, modular digital watch — 24-hour clock, stopwatch, and programmable alarm — implemented entirely in synthesizable Verilog HDL. The design cleanly separates concerns across eight modules: a combinational display decoder, a reusable debouncer, a clock divider, independent clock and stopwatch counters, a binary-to-BCD splitter, a buzzer driver, and a top-level module that ties them together through a simple counter-based mode state machine and a time-multiplexed display scan.

Beyond simply producing correct time values, the design addresses the practical realities of interfacing digital logic to the physical world: debouncing noisy mechanical switches, deriving multiple timing domains from a single oscillator, and sharing a limited set of output pins across four display digits through multiplexing. Together, these techniques make this project a compact but genuine example of RTL system design, and a solid foundation for further work such as FPGA deployment, additional alarm features, or a full calendar display.

Prepared by: Gayatri Galidevara